Revisiting the Compact Tail Assignment Model: Corrections and an Exact Event-Based Maintenance Reformulation

Viktor Borodin[a], Arun Kumar Pappala[b]

[a] *Taras Shevchenko National University, Kyiv, Ukraine*
[b] *Singapore University of Technology and Design, Singapore*

Abstract

This paper revisits the compact mixed-integer linear programming model for the Tail Assignment Problem (TAP) introduced by Khaled et al. (2018). We make three contributions. First, we show that the basic continuity and turn-time inequalities (C2–C4; constraints (3)–(6) in Khaled et al.) admit a simultaneous-departure corner case that is mathematically feasible but physically infeasible, and we close this gap with explicit non-overlap constraints (C5). Second, we identify and formally correct a flaw in the paper's cumulative-flight-time row (constraint (13) in Khaled et al.; not our aggregation constraint C11): the original formulation allows maintenance intervals to be incorrectly relaxed. We show that a two-row endpoint split is also insufficient and give an exact cumulative-state formulation. We give a numeric counterexample, evaluated against the real built constraint objects at two adversarial check patterns, in which the original and endpoint-split formulations each fail on a different pattern while the ordered formulation rejects both, and we independently verify every reported solution by replaying its maintenance counters outside the model. Third, we propose an alternative

to Khaled et al.'s set of constraints for the same TAP problem with type-A maintenance: an ordered, day-free event-based formulation that uses aircraft-local prefix state without a day-pair window. We prove the reformulation is correct and evaluate it alongside the two legacy rows and the strict corrected legacy variant. On the corrected feasible family, all four formulations achieve objective parity, while the event-based model uses fewer variables. The observed timing advantage is instance- and solver-specific. All code, instances, and result tables accompany the project.
Keywords: Tail assignment, Aircraft routing, Maintenance scheduling, Integer programming, Mixed-integer programming corrections

---

1. Introduction

The airline planning process is traditionally decomposed into a sequence of sub-problems: timetable design, fleet assignment, crew rostering, tail assignment, and recovery planning (Barnhart et al., 2003; Gopalan and Talluri, 1998). The *tail assignment problem* (TAP)—assigning a specific aircraft (identified by its tail number) to each flight leg—is the penultimate planning step and is tightly coupled with aircraft maintenance scheduling. Poor tail assignment decisions directly inflate operational costs through unnecessary ferry (repositioning) flights, suboptimal maintenance routing, and safety-margin violations.

In addition to the basic assignment and continuity requirements, the TAP must respect aircraft maintenance constraints. The most common maintenance type is *type-A*, which requires a maintenance check after a maximum flying-time interval. Type-A maintenance is typically performed at a lim-

ited set of airports, and the TAP must ensure that each aircraft is routed to a maintenance-capable airport before its flying-time limit is exceeded. The type-A maintenance requirement is a key operational constraint in the TAP and has been studied extensively in the literature.

Khaled et al. (2018) proposed a compact 0–1 linear programming formulation for the TAP without maintenance, featuring only $O(n_f n_p)$ binary variables (where n_f is the number of flights and n_p the fleet size). This is a significant reduction over the classic multi-commodity flow model of Levin (1971), which requires $O(n_f^2 n_p)$ variables. Further, Khaled et al. (2018) extended the model to include type-A maintenance requirements, which are defined by a maximum flying-time interval between maintenance checks, that adds other $O(n_f n_p D)$ binary variables, where D is the number of calendar days in the planning horizon. Computational experiments reported in that paper show that instances with up to 1 500 flight legs and 40 aircraft can be solved to certified optimality within one hour on a standard workstation.

Our contributions

The present paper makes three contributions that check and correct those results:

1. **Basic feasibility correction (C1–C4 / Khaled constraints (3)–(6)).** We show that the strict-time balance definition used in the compact basic model admits a simultaneous-departure corner case: one aircraft can be assigned to two flights departing at the same airport and same time without violating the balance inequalities. We formalize this counter-example and enforce physical executability by adding explicit pairwise non-overlap constraints (our C5).

Our non-overlap correction retains the $O(n_f n_p)$ binary-variable space but can add $O(n_f^2 n_p)$ conflict inequalities in the worst case. In practical timetables, only temporally conflicting pairs are instantiated; the resulting constraint count is therefore governed by the sparse conflict set $\mathcal{O}$ defined in Section 3.

2. **Cumulative-flight-time row correction.** We show that the cumulative-flight- time maintenance constraint (constraint (13) in Khaled et al. (2018)) contains a logical flaw in opening and one-endpoint intervals, and in the handling of consecutive flights and maintenance events. We also show by counterexample that a natural two-row endpoint split does not fully repair the model. We therefore formulate an exact cumulative-state recursion that propagates and resets the flight-hour counter without day-pair ambiguity.
3. **Event-based reformulation.** The day-indexed type-A model above still repeats a calendar-day index across every maintenance variable. We give a complete, day-free reformulation of the same type-A C1–C13 requirements, in which each flight's own timestamp supplies all calendar information and maintenance is represented as a virtual flight triggered by, and indexed on, the preceding real flight rather than by calendar day. This removes the day dimension from the maintenance model's size and lets the flight-hour counter propagate exactly along each aircraft's actual route, with no day-pair window and therefore no possibility of the split-form failure mode proved for the legacy cumulative-flight-time row.

Paper organisation

Section 2 reviews related work. Section 3 restates the basic TAP model and proves a C1–C4 corner-case correction closed by C1–C5. Section 4 presents the maintenance-extended model, the cumulative-flight-time row correction, and the induced interpretation updates for C8–C13 under the corrected basic model. Section 5 gives the day-free event-based reformulation of the type-A requirements and proves its correctness directly. Section 6 reports the numeric cumulative-flight-time counterexample and the four-method benchmarking results, and Section 7 concludes.

2. Literature Review

2.1. Mathematical programming formulations

Three main modelling paradigms underlie TAP research:

Set partitioning / column generation.. The non-compact formulation enumerates *strings* (feasible flight sequences per aircraft) as 0–1 variables. Because the number of strings is exponential, column generation is required (Lavoie et al., 1988; Minoux, 1984; Barnhart et al., 1998; Sarac et al., 2006). Combining column generation with branch-and-bound (*branch-and-price*) gives exact solutions but at significant algorithmic overhead. Robust extensions appear in Borndörfer et al. (2010) and Froyland et al. (2013).

Multi-commodity flow.. Levin (1971) introduced the compact multi-commodity flow formulation with $O(n_f^2 n_p)$ binary variables. This remains the standard benchmark for compact models and is the primary comparison point in Khaled et al. (2018).

Time-space networks.. Hane et al. (1995) proposed the time-space network for the *fleet assignment problem*. Nodes represent airport–time events; arcs encode ground waits, flights, and overnight stays. Extensions to aircraft routing appear in Clarke et al. (1996). Preprocessing by node aggregation and island isolation reduces model size (Hane et al., 1995; Sherali et al., 2006). Daily and weekly maintenance routing variants are studied in Liang et al. (2011) and Liang and Chaovalitwongse (2012); the latter also integrates fleet assignment decisions, but only approximate solutions are obtained via variable fixing.

Nonlinear / lifted formulations.. Haouari et al. (2012) present a compact formulation with nonlinear constraints that are linearised via the Reformulation–Linearisation Technique (RLT). The largest instances solved involve 344 flights, an order of magnitude below the instances handled by the model of Khaled et al. (2018).

2.2. MILP alternatives to the Khaled compact model

Beyond the compact binary assignment model of Khaled et al. (2018), several mixed-integer alternatives are relevant for exact TAP modelling and large-scale implementation:

- **Arc-flow MILP (time-indexed or event-indexed):** decision variables are attached to feasible flight-connection arcs instead of flight–aircraft assignment pairs, often yielding stronger network-structure constraints (Levin, 1971; Hane et al., 1995; Sherali et al., 2006).
- **Route-based master MILP (set partitioning):** aircraft duties are represented as columns in a master MILP, typically solved with branch-

and-price; this is exact but algorithmically heavier (Barnhart et al., 1998; Sarac et al., 2006; Klabjan, 2005).

- **Decomposed MILP frameworks:** Benders- or Dantzig–Wolfe-style decompositions separate assignment/routing from maintenance-resource feasibility, improving scalability on larger fleets (Cordeau et al., 2001; Mercier et al., 2005; Desaulniers et al., 1997).

- **Two-stage or rolling-horizon MILP:** first assign flights, then repair/optimize maintenance and turnaround feasibility in a second MILP, trading global optimality for tractable runtime (Clarke et al., 1996; Liang et al., 2011; Liang and Chaovalitwongse, 2012).

- **Robust/scenario MILP variants:** delay and disruption uncertainty is modelled through multiple scenarios with non-anticipativity-style linking constraints, producing schedules that are less brittle operationally (Borndörfer et al., 2010; Froyland et al., 2013).

In this taxonomy, the Khaled model is best viewed as a compact, directly solvable baseline that is simple to implement and benchmark, while the alternatives above trade model size, decomposition complexity, and robustness for stronger scalability or richer operational realism.

2.3. Comparison scope and fairness

For an empirical comparison to be methodologically fair, competing models should be evaluated under a common instance family, objective definition, maintenance semantics, and solver-budget protocol. In TAP, this condition is often not met across compact MILP, branch-and-price, Benders-decomposed,

and robust/scenario formulations because these approaches typically optimize different decision scopes or uncertainty assumptions. Accordingly, this paper uses Khaled et al. (2018) as the direct compact baseline for instance-matched comparisons, and treats other MILP families as complementary reference paradigms rather than strict one-to-one benchmarks.

2.4. Maintenance constraints

Maintenance scheduling within aircraft routing has been studied since the early work of Clarke et al. (1997). The column-generation framework of Barnhart et al. (1998) can embed type-A constraints through constrained shortest-path pricing. Cordeau et al. (2001) apply Benders decomposition to simultaneous routing and crew scheduling with non-linear maintenance resources. Lacasse-Guay et al. (2010) consider maintenance under different business-process models. We focus on the type-A flight-hour case and its exact event-based reformulation.

2.5. Heuristic approaches

Constructive heuristics for TAP are comparatively under-studied relative to exact methods. Grönkvist (2005) provides a comprehensive treatment including local-search improvements. Heuristic and metaheuristic solution methods are outside the scope of the present exact-formulation study and are not evaluated here.

2.6. Position of the present paper

The compact formulation of Khaled et al. (2018) represents the reference compact baseline for exact, cyclic-constraint-free TAP in this study. The

Table 1: Notation summary.

Symbol	Meaning
$\mathcal{F} = \{1, \ldots, n_f\}$	Set of flight legs
$\mathcal{P} = \{1, \ldots, n_p\}$	Set of aircraft (tail numbers)
$\mathcal{A} = \{1, \ldots, n_k\}$	Set of airports
$a_i^{\uparrow}$, $a_i^{\downarrow}$	Departure / arrival airport of flight i
$t_i^{\uparrow}$, $t_i^{\downarrow}$	Departure / arrival time of flight i (minutes)
d_i, $\bar{d}_i$	Departure / arrival day derived from flight times
Δt	Minimum ground turn time (minutes)
c_{ij}	Assignment cost of flight i to aircraft j
a_j^0	Initial position airport of aircraft j
$F_k^{\downarrow}(\theta)$	$\{i \in \mathcal{F} : a_i^{\downarrow} = k,\ t_i^{\downarrow} \leq \theta - \Delta t\}$
$F_k^{\uparrow}(\theta)$	$\{i \in \mathcal{F} : a_i^{\uparrow} = k,\ t_i^{\uparrow} < \theta\}$

present work builds directly upon it, providing: (i) a diagnosis of the C1–C13 formulation in Khaled et al. (2018) and an exact state-based replacement for the legacy cumulative-flight-time row, (ii) a new reformulation of the event-based model that resolves the counterexamples and improves computational performance, and (iii) an instance-matched benchmark of the implemented exact formulations.

3. The Basic Tail Assignment Model

We restate the compact model of Khaled et al. (2018) using the notation of Table 1. This section covers the model without maintenance constraints; they are introduced in Section 4.

3.1. Decision variables and objective

Let us define the decision variables and constraints for the basic tail assignment model.

C1 — Binary assignment domain.. The decision variable x_{ij} indicates whether flight i is assigned to aircraft j.

$$x_{ij} = \begin{cases} 1 & \text{if flight } i \text{ is operated by aircraft } j, \\ 0 & \text{otherwise,} \end{cases} \qquad i \in \mathcal{F},\ j \in \mathcal{P}. \tag{1}$$

The objective minimises total assignment cost:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij}. \tag{2}$$

3.2. Constraints

C2 — Flight coverage.. Each flight must be assigned to exactly one aircraft:

$$\sum_{j \in \mathcal{P}} x_{ij} = 1, \qquad \forall i \in \mathcal{F}. \tag{3}$$

C3,C4 — Equipment continuity (home/non-home airports).. For any aircraft j, any non-home airport $k \neq a_j^0$, and any flight i departing from k, aircraft j can be assigned to flight i only if it has already landed at k early enough, but at the home airport ($k = a_j^0$), one free unit is available at time zero

$$\sum_{i' \in F_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in F_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \ \geq \ x_{ij}, \qquad \forall j \in \mathcal{P},\ k \in \mathcal{A} \setminus \{a_j^0\},\ i \in \mathcal{F} : a_i^{\uparrow} = k. \tag{4}$$

$$\sum_{i' \in F_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in F_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \ \geq \ x_{ij} - 1, \qquad \forall j \in \mathcal{P},\ k = a_j^0,\ i \in \mathcal{F} : a_i^{\uparrow} = k. \tag{5}$$

3.3. Validity

Constraints (4)–(5) are intended to encode both equipment continuity (C2 of Khaled et al. (2018)) and turn-time feasibility (constraint (6) of that paper). The set $F_k^{\downarrow}(t_i^{\uparrow})$ counts only arrivals at least Δt minutes before the departure of flight i, so a prior arrival can supply flight i only after the required turn time. This mechanism is valid when departures are strictly ordered, but it does not prevent two equal-time departures from using the same available aircraft, as shown below.

Proposition 1 (Claimed in Khaled et al. 2018)**.** *Any feasible solution to* (1)–(5) *corresponds to a feasible flight plan satisfying the original continuity and turn-time conditions.*

The proposition is not valid as stated when distinct flights may have equal departure times. The following counterexample identifies the missing case.

3.4. Counter-example with simultaneous departures

The balance constraints (4)–(5) use $F_k^{\uparrow}(\theta) = \{i \in \mathcal{F} : a_i^{\uparrow} = k,\ t_i^{\uparrow} < \theta\}$ with a strict inequality in departure time. If two flights leave the same airport at the same time, neither flight appears in the other's departure-balance term. This can make an assignment feasible while being physically impossible as a flight plan.

Example 1 (Feasible to (1)–(5), infeasible in practice). Consider one aircraft j with initial airport $a_j^0 = \mathrm{B}$ and turn time $\Delta t = 30$ minutes. Let the three

flights be:

$$\begin{aligned} i_0 : \mathrm{B} \to \mathrm{A}, \quad & t^{\uparrow}_{i_0} = 500,\ t^{\downarrow}_{i_0} = 550, \\ i_1 : \mathrm{A} \to \mathrm{B}, \quad & t^{\uparrow}_{i_1} = 600,\ t^{\downarrow}_{i_1} = 700, \\ i_2 : \mathrm{A} \to \mathrm{C}, \quad & t^{\uparrow}_{i_2} = 600,\ t^{\downarrow}_{i_2} = 700. \end{aligned}$$

Set $x_{i_0 j} = x_{i_1 j} = x_{i_2 j} = 1$.

Coverage (3) is satisfied (single aircraft, all flights assigned). Flight i_0 satisfies the home-airport constraint (5), because no flight has arrived at or departed from B before time 500. For i_1 at the non-home airport A:

$$F^{\downarrow}_{\mathrm{A}}(t^{\uparrow}_{i_1}) = \{i_0\}, \qquad F^{\uparrow}_{\mathrm{A}}(t^{\uparrow}_{i_1}) = \varnothing,$$

because i_0 departs from B and $t^{\uparrow}_{i_2} = 600 \not< 600$. Hence constraint (4) gives

$$\sum_{i' \in F^{\downarrow}_{\mathrm{A}}(t^{\uparrow}_{i_1})} x_{i' j} - \sum_{i' \in F^{\uparrow}_{\mathrm{A}}(t^{\uparrow}_{i_1})} x_{i' j} = 1 - 0 = 1 \geq x_{i_1 j} = 1.$$

Symmetrically for i_2, $t^{\uparrow}_{i_1} = 600 \not< 600$, so the same calculation gives $1 \geq x_{i_2 j} = 1$. Therefore (1)–(5) are feasible.

However, the resulting plan is physically infeasible: one aircraft cannot operate both i_1 and i_2 simultaneously at time 600.

Implication.. In implementations, this corner case is eliminated by adding pairwise non-overlap constraints for time-overlapping flights on the same aircraft, $x_{i_1 j} + x_{i_2 j} \leq 1$.

Corrected C1–C5 feasibility criterion.. Define $\mathcal{O}$ as the set of flight pairs that cannot be operated by one aircraft because their operating intervals, including the turn-time buffer, overlap:

Conflict-set definition..

$$\mathcal{O} = \left\{ \{i_1, i_2\} \subseteq \mathcal{F} : [t^{\uparrow}_{i_1}, t^{\downarrow}_{i_1} + \Delta t) \cap [t^{\uparrow}_{i_2}, t^{\downarrow}_{i_2} + \Delta t) \neq \varnothing \right\}. \tag{6}$$

For every pair in this conflict set, impose

C5 — Pairwise non-overlap..

$$x_{i_1 j} + x_{i_2 j} \leq 1, \qquad \forall \{i_1, i_2\} \in \mathcal{O},\ \forall j \in \mathcal{P}. \tag{7}$$

The corrected core model is therefore (1)–(7). This preserves the original $n_f n_p$ binary-variable space while restoring physical executability. It does not, in general, preserve the linear constraint count: the overlap family contains $|\mathcal{O}| n_p$ inequalities.

3.5. Model size

Proposition 2 (Analogue of Khaled et al. 2018, Proposition 2)**.** *The original model* (2)–(5) *with binary* x *contains* $n_f n_p$ *binary decision variables and* $O(n_f n_p)$ *constraints, while the corrected model has* $n_f n_p$ *binary variables and* $O(n_f n_p + |\mathcal{O}| n_p)$ *constraints.*

The corrected model retains $n_f n_p$ binary variables and has $O(n_f n_p + |\mathcal{O}| n_p)$ constraints. In the worst case $|\mathcal{O}| = O(n_f^2)$, although timetable sparsity normally makes the conflict set much smaller. The original variable count compares favourably with Levin's $O(n_f^2 n_p)$-variable model; Khaled et al. (2018) also report that the time-space network formulation uses approximately 1.25–2 times as many variables and constraints on their test instances.

3.6. Corrected proposition and its proof.

The issue with the proof of feasibility in the original model is that it does not account for simultaneous departures of overlapping flights. Moreover, the logic of the published proof goes in the wrong direction, and instead of proving that any solution of C1–C4 defines correct feasibility, it proves that any feasible assignment flights satisfy C1-C4.

The corrected proposition and its accompanying proof look like the following:

Proposition 3 (Proposition 1)**.** *Any feasible solution to* (1)–(7) *corresponds to a feasible flight plan satisfying the original continuity and turn-time conditions.*

Proof. Let's formally prove it by induction on the number of planes. For the base case of one plane, any feasible assignment trivially satisfies the continuity and turn-time conditions:

- The single plane can start only for the airport it is assigned form the start, which is follows form the condition 5. Let's define this assigned flight as f_1, so the time of departure and arrival are $t_{\text{dep}}(f_1)$ and $t_{\text{arr}}(f_1)$, respectively.

- Any next k-th flight of this plane with time of departure $t_{\text{dep}}(f_k)$ must satisfy the turn-time condition $t_{\text{dep}}(f_k) \geq t_{\text{arr}}(f_{k-1}) + \tau$, where τ is the required turn time. This follows directly from the turn-time condition (4) and overlap constraints (7).

This completes the base case.

For the induction step, assume that any feasible assignment for $n-1$ planes satisfies the continuity and turn-time conditions. Consider a feasible assignment for n planes. Select one plane and consider its sequence of assigned flights. By the same argument as in the base case, this plane's flights satisfy the continuity and turn-time conditions. Removing this plane from the assignment leaves a feasible assignment for the remaining $n-1$ planes, which by the induction hypothesis satisfies the continuity and turn-time conditions. Therefore, the entire assignment for n planes satisfies the continuity and turn-time conditions. This completes the induction step. □

4. The Maintenance-Extended Model and Cumulative-Flight-Time Correction

4.1. Maintenance problem definition

Following Khaled et al. (2018), we consider type-A checks triggered after at most $T^A_{\max}$ cumulative flight hours. Checks take place at night after the last flight on a given day in an airport with maintenance capacity.

Additional notation is collected in Table 2.

4.2. Additional decision variables

C6, C7 — Boolean variables for maintenance..

$$z_{ijd} = \begin{cases} 1 & \text{night maintenance at arrival airport of } i,\ \text{aircraft } j,\ \text{day } d, \\ 0 & \text{else;} \end{cases}$$

$$y_{jd} = \begin{cases} 1 & \text{maintenance of aircraft } j \text{ on day } d, \\ 0 & \text{else,} \end{cases}$$

Table 2: Additional notation for the maintenance model.

Symbol	Meaning
$\mathcal{D} = \{1, \ldots, n_d\}$	Planning days
d_i	Departure day of flight i
$\bar{d}_i$	Arrival day of flight i
$\mathcal{M} \subseteq \mathcal{A}$	Maintenance-capable airports
$\text{mcap}_{m,d}$	Maintenance capacity of airport m on day d
mc_{ijd}	Cost of night maintenance at arrival airport of flight i, aircraft j, day d
$F_d^{\downarrow}(m)$	Flights arriving at airport m on day d
$F(d, i)$	$\{i' : d_{i'} = d,\ t_{i'}^{\uparrow} > t_i^{\downarrow}\}$
$T_{\max}^{A}$	Max cumulative flight minutes before type-A check
$d_{\max}$	Max calendar-day interval between checks
$M_{dd'}$	Big-M for day pair (d, d')
$F_{d,d'}$	Flights with departure day in $(d, d']$

The variable z_{ijd} is instantiated only when flight i arrives at a maintenance-capable airport on day d. Thus its admissible domain is $i \in \bigcup_{m \in \mathcal{M}} F_d^{\downarrow}(m)$. We assume in this type-A model that a night check fits within the overnight ground period.

4.3. Extended model

The objective becomes:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij} + \sum_{d \in \mathcal{D}} \sum_{m \in \mathcal{M}} \sum_{i \in F_d^{\downarrow}(m)} \sum_{j \in \mathcal{P}} mc_{ijd}\, z_{ijd}. \tag{8}$$

Constraints (1)–(7) are retained. Additional constraints:

C8 — Maintenance blocks same-day subsequent flights..

$$z_{ijd} + x_{i'j} \leq 1, \quad \forall d \in \mathcal{D},\ m \in \mathcal{M},\ i \in F_d^{\downarrow}(m),\ i' \in F(d,i),\ j \in \mathcal{P}. \tag{9}$$

C9 — Maintenance requires assignment..

$$x_{ij} \geq z_{ijd}, \quad \forall d \in \mathcal{D},\ m \in \mathcal{M},\ i \in F_d^{\downarrow}(m),\ j \in \mathcal{P}. \tag{10}$$

C10 — Capacity..

$$\sum_{i \in F_d^{\downarrow}(m)} \sum_{j \in \mathcal{P}} z_{ijd} \leq \mathrm{mcap}_{m,d}, \qquad \forall d \in \mathcal{D},\ m \in \mathcal{M}. \tag{11}$$

C11 — Aggregation ($z \to y$)..

$$\sum_{m \in \mathcal{M}} \sum_{i \in F_d^{\downarrow}(m)} z_{ijd} = y_{jd}, \qquad \forall d \in \mathcal{D},\ j \in \mathcal{P}. \tag{12}$$

C12 — Maximum-spacing constraint..

$$\sum_{r \in [d,d']} y_{jr} \geq 1, \qquad \forall d, d' \in \mathcal{D} \text{ s.t. } d' - d = d_{\max} - 1,\ \forall j \in \mathcal{P}. \tag{13}$$

For the type-A-only model studied in this paper, C12 is not an additional calendar-spacing requirement: type-A maintenance is governed by the exact flight-hour state introduced in the cumulative-state formulation below. The displayed row is retained as the generic day-indexed maintenance notation and becomes active only for a calendar-time maintenance type, which is outside the scope of this paper.

The original paper's cumulative-flight-time constraint is C13:

C13 — Paper-based cumulative-flight-time constraint..

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \ \leq\ T^A_{\max} + M_{dd'}\Big(2 - y_{jd} - y_{jd'}\Big) + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{14}$$

for all $j \in \mathcal{P}$, all pairs (d, d') with $2 \leq d' - d \leq d_{\max} - 1$, $d < n_d$, where $\tilde{t}_i$ is the flight duration of leg i.

4.4. Impact of corrected C1–C5 feasibility on C8–C13

The correction introduced in Section 3 (non-overlap constraint (7)) changes the admissible assignment set for x, and therefore the interpretation of all maintenance constraints layered on top of x.

C8.. No algebraic change is required in (9); however, with (7) active, the blocking relation now operates on physically realizable trajectories only. This removes spurious cases where maintenance placement was evaluated on infeasible simultaneous departures.

C9–C12.. Constraints (10)–(13) remain structurally unchanged. Their feasible region is tightened indirectly because z and y are linked to a strictly smaller, physically valid x-space.

C13 and cumulative-hour rows.. Hour-accumulation constraints are evaluated over a physically executable x-space once (7) is imposed. This correction is necessary, but it does not by itself establish that a particular cumulative-hour algebraic form correctly propagates the maintenance counter; that property is analysed below.

In summary, the C1–C5 correction primarily changes the domain over which the maintenance constraints operate. It is independent of the separate algebraic correction required for cumulative-hour propagation.

4.5. Issues of day-indexed maintenance constraints C8–C13

Usage of day-indexed maintenance variables y_{jd} and z_{ijd} is operationally intuitive, gives clear modeling semantics, but it has the following drawbacks:

- The day-indexed formulation requires a large number of big-M constants $M_{dd'}$ to cover all day pairs (d, d') with $2 \leq d' - d \leq d_{\max} - 1$. This is a combinatorial number of rows, and the big-M values are necessarily loose because they must cover all possible flight-hour accumulations over the horizon.

- The day-indexed formulation does not propagate the flight-hour state exactly. The big-M relaxation allows the flight-hour counter to be reset prematurely, leading to potential violations of the intended maintenance intervals. For example, if a maintenance night is scheduled on day d and the next maintenance night is scheduled on day d', the big-M constraints only enforce that the cumulative flight hours between d and d' do not exceed $T^A_{\max}$, but they do not prevent excessive flight hours before day d.

- The day-indexed formulation forces a specific maintenance schedule on each day for a given time zone, thus allowing to have two different maintenance checks within 24 hrs, but not in two different days within the same 24-hr period.

- The day-indexed formulation does not allow for a natural integration with the ordered flight formulation, which is more efficient and compact. The ordered flight formulation allows for a more direct representation of the maintenance events in relation to the flight sequence, leading to a more efficient model.

- The original cumulative-flight-time row (paper equation) in the day-indexed formulation is flawed, as it does not correctly enforce the flight-

hour limit between two consecutive maintenance nights. This flaw can lead to infeasible solutions that violate the intended maintenance intervals.

4.6. Legacy cumulative-flight-time row: original formulation and flaw

The original paper's cumulative-flight-time constraint (14) is intended to enforce that the cumulative flight hours between two consecutive maintenance nights do not exceed $T^A_{\max}$. However, as shown in Lemma 1, this constraint can fail to enforce the intended maintenance intervals in certain cases, particularly when there is no maintenance scheduled at the start of the interval.

Lemma 1 (Flaw in (14))**.** *If $y_{jd} = 0$ and $y_{jd'} = 1$, with no intermediate check, the RHS of* (14) *becomes $T^A_{\max} + M_{dd'}$. Because $M_{dd'}$ is chosen as a large upper bound on cumulative flight time, the row is then vacuous and imposes no limit on flying time before the first check of the period. The same issue occurs symmetrically when $y_{jd} = 1$ and $y_{jd'} = 0$.*

Proof. Direct substitution of $y_{jd} = 0$, $y_{jd'} = 1$, $\sum_r y_{jr} = 0$ into (14) gives the stated RHS. Since $M_{dd'}$ is chosen as the maximum possible cumulative flight time over the horizon (a large positive constant), the resulting constraint is trivially satisfied regardless of the actual flight hours flown. □

4.7. Legacy cumulative-flight-time row: endpoint-split strengthening and limitation

An endpoint-split strengthening considered for replacing (14) consists of two constraints, each relaxed by one endpoint:

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\, y_{jd} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{15}$$

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\, y_{jd'} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{16}$$

for all $j \in \mathcal{P}$, $(d, d') \in \mathcal{D}^2$ with $2 \le d' - d \le d_{\max} - 1$, $d < n_d$.

Lemma 2 (The endpoint split is not sufficient)**.** *With $F_{d,d'}$ defined as the flights departing in $(d, d']$, constraints* (15)–(16) *do not necessarily bound all flying time between two consecutive maintenance nights.*

Proof. Let $T^A_{\max} = 10$, $M_{dd'} = 20$, $d_{\max} \ge 4$, and consider days $1, \dots, 4$ with $y_{j1} = y_{j4} = 1$ and $y_{j2} = y_{j3} = 0$. Assign 10 units of flying time on day 2 and 10 units on day 4. For $(d, d') = (1, 3)$, the accumulated time is 10 and the second split inequality is active. For $(2, 4)$, the accumulated time is 10 and the first split inequality is active. For $(1, 4)$, the accumulated time is 20, but both inequalities have right-hand side $T^A_{\max} + M_{14} = 30$ because both endpoints contain checks. Thus every split inequality is satisfied although 20 units are flown between the consecutive checks, exceeding the limit 10. □

The original and split formulations are therefore not ordered by strength: the original row is active when both endpoints contain checks, whereas the split rows are active when at least one corresponding endpoint indicator is zero. Neither pattern alone gives an exact cumulative counter.

4.8. Exact cumulative-state formulation for cumulative flight hours

Define the flying time assigned to aircraft j on day d by

C13a Daily flight time.

$$v_{jd} = \sum_{i \in \mathcal{F}:\, d_i = d} \tilde{t}_i x_{ij}. \tag{17}$$

Let $h_{jd} \geq 0$ be the accumulated type-A flying time immediately after the flights of day d and before maintenance that night. Set $h_{j0} = \Phi_j^A$ and $y_{j0} = 0$. For consecutive days, impose

C13b-C13f Accumulated flight time constraints.

$$h_{jd} \geq h_{j,d-1} + v_{jd} - M^H y_{j,d-1}, \tag{18}$$

$$h_{jd} \leq h_{j,d-1} + v_{jd} + M^H y_{j,d-1}, \tag{19}$$

$$h_{jd} \geq v_{jd}, \tag{20}$$

$$h_{jd} \leq v_{jd} + M^H (1 - y_{j,d-1}), \tag{21}$$

$$0 \leq h_{jd} \leq T^A_{\max}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D}, \tag{22}$$

where $M^H \geq T^A_{\max}$ is valid because $h_{j,d-1} \leq T^A_{\max}$. If $y_{j,d-1} = 0$, the first two inequalities give $h_{jd} = h_{j,d-1} + v_{jd}$. If $y_{j,d-1} = 1$, the reset inequalities give $h_{jd} = v_{jd}$. The upper bound then enforces the flight-hour threshold on every day, including the flights operated immediately before a night check.

4.9. Equivalent implementation strengthening

The exact-state formulation admits a smaller, tighter implementation without changing its integer feasible region. The strengthened implementation is type-A-only, matching the scope of this paper. Its state variable receives the threshold directly as a variable bound, $0 \leq h_{jd} \leq T^A_{\max}$. The explicit copy of this upper bound in (22) is then unnecessary in the generated matrix.

Define the aircraft-day upper bound

$$\bar{v}_{jd} = \sum_{i \in \mathcal{F}:\, d_i = d,\, (i,j) \in \mathcal{X}} \tilde{t}_i \tag{23}$$

and the reachable state bounds

$$U_{j0} = \min\{T^A_{\max}, \Phi^A_j\}, \qquad U_{jd} = \min\{T^A_{\max}, U_{j,d-1} + \bar{v}_{jd}\}. \tag{24}$$

Because $h_{jd} \le U_{jd}$ is valid by induction, the reset linearization for day d may use $U_{j,d-1}$ instead of the horizon-wide $T^A_{\max}$. This is particularly tighter near the beginning of the planning horizon.

Finally, the strengthened implementation generates maximal interval-clique rows for flight overlap and omits the pairwise rows implied by those cliques. Every conflicting pair in an interval graph belongs to a clique, so this replacement preserves the integer feasible set while reducing duplicate rows. The implementation is `LegacyCorrectedStrengthenedMILPScheduler`; the original `LegacyCorrectedMILPScheduler` remains unchanged for reproduction of the four-method baseline.

4.10. Pre-horizon accumulated hours

The original paper does not account for flying time accumulated before the planning horizon. In the cumulative-state formulation this history is included exactly through the initial condition

$$h_{j0} = \Phi^A_j, \qquad 0 \le \Phi^A_j \le T^A_{\max}, \quad \forall j \in \mathcal{P}. \tag{25}$$

The condition is required for every aircraft, including $\Phi^A_j = 0$; no separate opening-window big-M inequality is needed.

4.11. Integrality constraints

$$x_{ij} \in \{0,1\}, \quad z_{ijd} \in \{0,1\}, \quad y_{jd} \in \{0,1\}, \qquad \begin{aligned} &(i,j) \in \mathcal{F} \times \mathcal{P}, \\ &d \in \mathcal{D}, \quad i \in \bigcup_{m \in \mathcal{M}} F_d^{\downarrow}(m) \text{ for } z_{ijd}. \end{aligned} \tag{26}$$

The variables h_{jd} are continuous and satisfy $0 \leq h_{jd} \leq T^A_{\max}$ as imposed in (22).

As we see, the fixes to the original paper's maintenance model are twofold: (i) the C0–C5 correction, which ensures that the flight assignment is physically feasible, and (ii) the cumulative-flight-time row correction. Also, the cumulative-flight-time correction is implemented in two ways: (i) the endpoint-split strengthening, which is a partial fix, and (ii) the exact cumulative-state formulation, which is a complete fix. The exact cumulative-state formulation correctly propagates the flight-hour state and enforces the maintenance intervals as intended. The complexity of the corrected model is higher than the original model, because we need to track the cumulative flight hours explicitly that require more $O(n_f D)$ variables, but it is necessary to ensure that the solutions are physically feasible and respect the maintenance requirements. day-indexed model is higher than the original model, because the cumulative flight-hour state h_{jd} adds $O(n_p n_d)$ continuous variables,

The additional complexity is justified by the need to accurately model the maintenance requirements and ensure that the resulting flight schedules are feasible in practice.

4.12. Rigorous feasibility theorem for C1–C13f

We now state the corrected sufficiency result for the type-A maintenance model. Consider the constraint family (1)–(7), (9)–(12), (18)–(22), and the stated variable domains. Constraint C12 is inactive for the type-A-only model: hour spacing is enforced by the cumulative-state rows rather than by calendar-day covering.

Theorem 1 (Physical feasibility under corrected C1–C13f)**.** *Any solution satisfying the above constraints, with x, z, and y binary and h continuous, induces for each aircraft j a physically executable trajectory over the planning horizon: flights assigned to j are temporally non-overlapping, satisfy airport continuity and turn time, and admit a day-by-day maintenance schedule satisfying capacity and hour-spacing rules.*

Proof. Fix an aircraft j and let $S_j = \{i \in \mathcal{F} : x_{ij} = 1\}$ ordered by nondecreasing departure time.

(i) Flight assignment and continuity. By (3), every flight is assigned to exactly one aircraft. By (4)–(5), each departure assigned to j has sufficient prior balance at its departure airport, with the initial unit at a_j^0. Therefore airport continuity holds along S_j.

(ii) Temporal executability. For any overlapping pair $\{i_1, i_2\} \in \mathcal{O}$, (7) enforces $x_{i_1 j} + x_{i_2 j} \leq 1$. Hence no two flights assigned to j overlap once turn time is included. Combined with (i), this yields a realizable single-aircraft flight timeline.

(iii) Maintenance event consistency. By (10), maintenance trigger variables can only be activated on assigned flights. By the aggregation equality (12), each day's maintenance trigger is represented by the single aggregate

indicator y_{jd}; by the binary day indicator in (12), the maintenance triggers on day d are represented by the corresponding aggregate maintenance state.

(iv) Maintenance capacity and blocking. By (11), airport-day capacities are respected. By (9), if maintenance is triggered after flight i on day d, then any subsequent same-day flight is forbidden for the same aircraft. Under the stated overnight-duration assumption, the maintenance action is operationally executable.

(v) Hour/day maintenance admissibility. By (18)–(22), the cumulative flight-time state increases by the flying time assigned each day, resets after a maintenance night, and never exceeds $T^A_{\max}$.

Steps (i)–(v) show that every aircraft has a consistent, non-overlapping flight sequence with feasible maintenance placement and resource usage. Therefore the global solution is physically feasible. □

5. Ordered Event-Based Reformulation for a Flight-hour Maintenance

In this paper, we present an ordered event-based model for flight-hour maintenance. It removes day-indexed maintenance decision states and does not introduce the standalone successor-arc event formulation. Flights are processed in nondecreasing departure-time order, and maintenance is represented by a binary event attached to the flight that triggers it. Thus the design-note variables x_{ij} and z_{ij} are represented below through an ordered position index r that maps to a flight index i_r.

5.1. Ordered event representation

For flight i, let o_i and a_i be its departure and arrival airports, p_i its flying time, and $t_i^{\uparrow}$ and $t_i^{\downarrow}$ its departure and arrival timestamps (the same macros as in Section 3). For aircraft j, let a_j^0 be its initial airport.

Define the ordered-position index set

$$\mathcal{R} = \{1, \ldots, n_f\},$$

and let $(i_1, \ldots, i_{n_f})$ be flights sorted by $(t_i^{\uparrow}, i)$. Position $r \in \mathcal{R}$ refers to flight i_r. This is the only role of r: it is an order index over flights. Define

$$\mathcal{R}^M = \{r \in \mathcal{R} : i_r \in \mathcal{F}^M\},$$

and let $\mathcal{Z}$ be the set of admissible event pairs (r, j) for which flight i_r can trigger the single type-A check on aircraft j. Use the following A-only variables:

E1-E3. Ordered event variables..

- $x_{rj} \in \{0, 1\}$: aircraft j operates flight i_r;
- $z_{rj} \in \{0, 1\}$: a type-A maintenance event is performed after flight i_r on aircraft j;
- $h_{rj} \geq 0$: cumulative type-A flight time of aircraft j immediately after processing position r.

Because $r \leftrightarrow i_r$ is one-to-one, the relation to the original notation is direct: $x_{rj} = x_{i_r j}$ and $z_{rj} = z_{i_r jd}$ for the corresponding arrival day d; the check type is always A in this section.

A maintenance event after flight i_r is assignment-linked and starts at its arrival time. The corrected airport-continuity, turn-time, and overlap constraints remain active on the assignment variables. In particular, an assignment is allowed only when it can be connected to the aircraft's preceding assigned flight and when the required maintenance duration leaves enough time before the next assigned departure.

The objective is

$$\min \sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{P}} c_{rj}x_{rj} + \sum_{r\in\mathcal{R}^M}\sum_{j\in\mathcal{P}} mc_{rjA}\, z_{rj}. \tag{27}$$

The event formulation numbers its route constraints independently as follows: E4 is flight coverage, E5 is non-home continuity, E6 is home-airport continuity and turn-time balance, and E7 is pairwise non-overlap. These are implemented by `_add_c1_coverage`, `_add_c23_turn`, and `_add_overlap`. E1–E3 are the binary/continuous variable domains defined above.

The event constraints separate maintenance blocking (E14), event assignment (E8), and maintenance capacity (E15).

E8. Event assignment.. Events satisfy

$$z_{rj} \le x_{rj}, \qquad \forall (r,j) \in \mathcal{Z}, \tag{28}$$

The integer event-count link in (43) is an auxiliary C11 aggregation, not a separate E family. Timestamp-based incompatibility rows are E14; they prevent an event from being followed by a departure before the maintenance duration and turn time have elapsed.

5.2. Prefix-state propagation

Let $T_F := T^A_{\max}$ be the maximum flying time between type-A maintenance events and let Φ_j be the pre-horizon accumulated flying time of aircraft j. At every ordered position, an unassigned flight carries the state forward, an assigned flight adds its duration, and an event resets the state after its triggering flight. Using valid position-specific big-M values M_{rj}, the transition is represented by

E9–E13. Prefix-state propagation..

$$h_{rj} \geq h_{r-1,j} - M_{rj}x_{rj}, \tag{29}$$

$$h_{rj} \leq h_{r-1,j} + M_{rj}x_{rj}, \tag{30}$$

$$h_{rj} \geq h_{r-1,j} + p_r - M_{rj}(1 - x_{rj}) - M_{rj}z_{rj}, \tag{31}$$

$$h_{rj} \leq h_{r-1,j} + p_r + M_{rj}(1 - x_{rj}) + M_{rj}z_{rj}, \tag{32}$$

$$h_{rj} \leq T_F(1 - z_{rj}) \tag{33}$$

The first two rows (E9–E10) carry the state at an unassigned position. The next two rows (E11–E12) impose accumulation for an assigned flight unless an event is selected. The final row (E13), together with the nonnegative state domain, gives the exact post-event state: $h_{rj} = 0$ when $z_{rj} = 1$, because the maintenance occurs after flight i_r. The final row together with (E3) bounds the state in all cases and prevents a maintenance event from repairing an arrival that already exceeds T_F. The initial state is $h_{0j} = \Phi_j$.

5.3. Maintenance blocking and capacity constraints

Let's assume that every maintenance event starts immediately after its triggering flight. Let δ_A be the duration of the type-A check and let Δt be

the minimum post-maintenance turn time.

E14. Maintenance blocking (C8).. For an event (r, j), define the incompatible following-flight set

$$F_r^{\text{dep}} = \left\{ \ell \in \mathcal{F} : t_\ell^{\uparrow} < t_r^{\downarrow} + \delta_A + \Delta t \right\}. \tag{34}$$

If $z_{rj} = 1$, every selected incompatible flight is blocked:

$$x_{\ell j} + z_{rj} \leq 1, \qquad \forall \ell \in F_r^{\text{dep}},\ j \in \mathcal{P}. \tag{35}$$

Only feasible (ℓ, j) assignment arcs are generated.

This constraint also can be rewritten in the next equivalent form:

$$M(x_{r'j} - 1) + t_j^{\uparrow} \geq t_r^{\downarrow} z_{rj} + \delta_A + \Delta t, \quad \forall (r', j) \in \mathcal{F} \times \mathcal{P},\ r' > r. \tag{36}$$

E15. Maintenance capacity (C10).. Define the active event set

$$\mathcal{E}(a, r) = \left\{ (i, j) : a_i^{\downarrow} = a, \quad t_r^{\downarrow} \leq t_i^{\uparrow} < t_r^{\downarrow} + \delta_A \right\}. \tag{37}$$

At every breakpoint, active maintenance occupancy satisfies

$$\sum_{(i,j) \in \mathcal{E}(a,r)} z_{ij} \leq \text{mcap}_a, \qquad \forall a \in \mathcal{M},\ r \in \mathcal{R}. \tag{38}$$

This constraint ensures that the number of simultaneously active maintenance events at any airport does not exceed the airport's maintenance capacity for every $r \in \mathcal{R}$.

5.4. Complete E1–E15 inventory of the implemented A-only model

The following list gives every event-family component constructed by the current ordered A-only builder. Legacy C labels are given only as cross-references.

E1–E3: variable domains. The variables x_{rj} and z_{rj} are binary and h_{rj} is nonnegative. These domains correspond to the assignment, event, and prefix-state variables.

E4: flight coverage (legacy C2). Each ordered flight is assigned exactly once:

$$\sum_{j \in \mathcal{P}} x_{rj} = 1, \qquad r \in \mathcal{R}. \tag{39}$$

E5–E6: aircraft continuity and turn time (legacy C3–C4). The corrected equipment-flow balance is applied to x_{rj}, including the initial aircraft position and the minimum turn time. The implementation is `e5_e6_continuity_turn`.

E7: pairwise non-overlap (legacy C5). For every temporally conflicting flight pair and aircraft, the builder adds

$$x_{ij} + x_{\ell j} \leq 1. \tag{40}$$

These rows are constructed by `e7_pairwise_overlap`; in the current ground-truth formulation they are the corrected C5 family.

The current ground-truth Sections 01–04 do not define a separate clique constraint number; any clique strengthening is an implementation strengthening of C5 rather than a new numbered requirement.

E8: event assignment (legacy C6–C7 and C9). The maintenance event variables z_{rj} are binary and are instantiated only for flights arriving at a maintenance-capable airport. There is no independent daily y variable in this formulation. Each admissible arrival can trigger at most one type-A event for an aircraft.

E14: maintenance blocking (legacy C8). If $z_{rj} = 1$, every selected flight whose departure falls before the event's completion plus turn time is blocked:

$$x_{\ell j} + z_{rj} \leq 1. \tag{41}$$

Only economically relevant timestamp conflicts are generated.

E15: maintenance capacity (legacy C10). At every airport timestamp breakpoint, the active maintenance events satisfy

$$\sum_{(r,j)\in\mathcal{E}(a,\theta)} z_{rj} \leq \mathrm{mcap}_a, \tag{42}$$

where $\mathcal{I}(a,\theta)$ contains events whose maintenance intervals cover r. This is implemented by the E15 component `e15_maintenance_capacity`; it is the maintenance-capacity constraint that was not explicitly displayed earlier.

Auxiliary C11 event-count aggregation. The binary day indicator is intentionally not retained. Define the nonnegative integer event count n_{jd} by

$$n_{jd} = \sum_{r:\, \lfloor t^{\downarrow}_{i_r}/1440 \rfloor = d} z_{rj}, \qquad n_{jd} \in \mathbb{Z}_+. \tag{43}$$

This preserves C11's aggregation information without imposing the removed one-event-per-day restriction.

Inactive legacy C12. Calendar spacing is inactive in the A-only model. It is implemented only by the C/D calendar extension, which is not enabled here.

E9–E13: cumulative flight-hour state (legacy C13–C13f). The paper's C13 row is replaced by the auxiliary prefix state h_{rj}, which is nonnegative, carries forward at unassigned positions, accumulates an assigned flight when $z_{rj} = 0$, resets to zero when $z_{rj} = 1$, and is bounded by $T^A_{\max}$. The daily flight-time definition is C13a and the state rows are C13b–C13f. These are implemented by `event_hour_state`; they replace the legacy day-pair C13 row rather than adding another numbered C-constraint.

The event formulation therefore has no additional business rule hidden under an unnumbered C-constraint. The auxiliary families are the integer C11 event counts and the linearization rows for the C13 hour state. The airport occupancy quantity $q_{a,j}$ proposed in the design note is not introduced as a variable: its role is represented directly by the C10 timestamp-capacity sums, which is smaller and equivalent for the A-only case.

5.5. Equivalent A-only implementation strengthening

The E1–E15 formulation also admits an equivalent reduced implementation. The daily event count in (43) is not used by any E-constraint or by the objective, so it can be recovered after solution rather than represented by integer variables and equalities. With only one maintenance type, event-type exclusivity is implied by the binary domain of z_{rj} and is omitted.

The prefix state receives its valid upper bound directly as a variable bound. Consequently, the additional generated row that repeats this bound is redundant with E13 and is removed. Identical E15 active-event sets are emitted only once, and a capacity row is omitted whenever its number of

binary terms does not exceed the capacity and is therefore satisfied for every assignment. Finally, maximal interval-clique rows replace the pairwise-plus-clique E7 implementation; every conflicting pair belongs to a maximal clique, so the integer feasible set is unchanged.

The baseline implementation remains `PaperEventBasedMILPScheduler`. The equivalent reduced implementation is `OptimizedPaperEventBasedMILPScheduler`; keeping both classes makes the effect of implementation choices independently reproducible.

Table 3 records the correspondence with the numbered day-indexed requirements. Some legacy rows disappear because their variables are eliminated; this is a formulation change, not an omitted feasibility condition.

Proposition 4 (Ordered prefix-state validity)**.** *Assume the corrected route constraints define a physically feasible integral assignment, every event is attached to an assigned flight at a maintenance-capable airport, and the supplied initial state is valid. Then the ordered prefix-state formulation satisfies the type-A flight-hour limit.*

Proof. Induct on the ordered flight positions. An unassigned position carries the state unchanged. An assigned position adds its flying time unless an event is selected, in which case the reset rows set the state to zero after the current flight. The final transition row and the upper state bound prevent the threshold from being exceeded before a reset. The induction starts from Φ_j, so every assigned route satisfies the type-A limit. □

Table 3: Coverage of the numbered C1–C13f requirements in the ordered A-only formulation.

Requirement	Day-indexed role	Ordered A-only counterpart
C1	Binary assignment domain	Binary domain on x_{rj}.
C2	Flight coverage	Coverage equation on x_{rj} (equivalently $x_{i_r j}$).
C3–C4	Airport continuity and turn time	Corrected equipment-flow balance on assignment variables.
C5	Pairwise non-overlap	Pairwise overlap inequalities on x.
C6–C7	Maintenance variables z, y	Event indicator z_{rj}; the daily indicator y is not a primitive variable.
C8	Maintenance blocks later flights	Timestamp blocking using $\delta_A + \Delta t$.
C9	Maintenance requires assignment	The event-assignment inequality above.
C10	Maintenance capacity	Direct timestamp-overlap capacity rows, equivalent to an airport occupancy state.
C11	Aggregate events by aircraft and day	Integer event count $n_{jd} = \sum_{r:\, d(i_r)=d} z_{rj}$.
C12	Calendar-time spacing	Inactive for type-A-only maintenance in this paper.
C13	Paper cumulative-flight-time row	Replaced by the exact ordered prefix state.
C13a	Daily assigned flight time	Implicit in the ordered flight sequence.
C13b–C13f	Exact cumulative-state recursion and bound	Ordered prefix-state rows (33).

5.6. Why the ordered formulation is the better method

5.7. Theoretical complexity and model size

Let $n_f = |\mathcal{F}|$, $n_p = |\mathcal{P}|$, and let n_m denote the number of flights that arrive at a maintenance-capable airport. In the A-only ordered model, the assignment variables contribute at most $n_f n_p$ binary variables, while the maintenance-event variables contribute at most $n_m n_p$ binary variables. The aircraft-local prefix state contributes at most $n_f n_p$ continuous variables; with sparse aircraft eligibility, all three counts are smaller and are determined by

the feasible (i, j) pairs. Thus the principal variable bound is

$$O(n_f n_p + n_m n_p),$$

with no additional day-indexed y_{jd} or occupancy-state $q_{a,j}$ family. The ordered implementation instead stores one state position per feasible flight–aircraft pair.

The main linear constraint families are $O(n_f)$ for coverage, $O(n_f n_p)$ for continuity and event assignment, and $O(n_f n_p)$ for the prefix-state equations. C11 aggregation contributes $O(n_p D)$ equalities for the event counts, without restricting their values. Pairwise non-overlap and timestamp blocking depend on the conflict sets and are $O(|\mathcal{O}|n_p)$ and $O(|\mathcal{B}|)$, respectively, where $\mathcal{O}$ is the flight-overlap set and $\mathcal{B}$ is the generated set of event–following-flight blocking pairs. Timestamp-capacity rows are generated per airport breakpoint, so their number is proportional to the number of distinct event timestamps rather than to a full time grid. In the worst case, $|\mathcal{O}| = O(n_f^2)$ and the sparse conflict families can also be quadratic, but timetable sparsity usually makes their observed counts much smaller. The theoretical conclusion is therefore precise: the Event-based model removes the day dimension from the principal variable families and keeps prefix-state size linear in feasible flight–aircraft pairs; it does not guarantee fewer total constraints unless the induced conflict and blocking sets are also sparse.

The ordered formulation combines event-triggered maintenance with a global flight order and aircraft-local prefix states. It removes the day-pair accumulation ambiguity of the legacy cumulative-flight-time row, avoids successor-variable expansion, and keeps the physical route constraints needed for valid tail assignments. For type-A maintenance, C12's calendar-spacing role is in-

active: the flight-hour limit is enforced by the exact prefix-state recursion. The computational study in Section 6 evaluates this formulation against the legacy model on matched horizon and fleet-size grids.

6. Computational Study

6.1. Implementation and environment

All models are implemented in Python using Pyomo. The reported exact solves use unrestricted IBM ILOG CPLEX 12.6 for AMPL through Pyomo's `cplexamp` interface. The deterministic instance generator and all benchmark scripts are included in the companion repository. Every reported solution is checked independently by replaying the aircraft route and its maintenance counter.

6.2. Cumulative-flight-time row verification

The original cumulative-flight-time row and the natural endpoint-split replacement were evaluated on the hand-built four-day counterexample `data/instances/c13_loopho`
The instance fixes one aircraft, $T^A_{\max} = 600$ minutes, $M_{14} = 1200$ minutes, 1,200 minutes of flying in $(1, 4]$, and two check patterns: checks at both endpoints, and a check only at the end. The actual production row objects admit complementary violating patterns: the original row is relaxed when only one endpoint is checked, while the split replacement is relaxed when both endpoints carry checks. The exact cumulative-state and ordered event-state formulations reject both patterns. This directly confirms the algebraic analysis in Sections 4.6 and 4.7.

Table 4: Four-way numeric verification of the cumulative-flight-time flaw. The fixed counterexample has $T^A_{\max} = 600$, $M_{14} = 1200$, and LHS $\sum_{i \in F_{1,4}} x_{ij} \tilde{t}_i = 1200$. "Accepts" means every row of that formulation is satisfied; "rejects" means the violation is detected.

Scenario	Khaled (13)	Legacy split	Legacy corrected	Event-based
Both endpoints checked	rejects	accepts (wrong)	rejects	rejects
Only end checked	accepts (wrong)	rejects	rejects	rejects

6.3. Four-method type-A comparison

The ordered implementation removes explicit successor variables and propagates an aircraft-local prefix state through flights sorted by departure time. A maintenance event is attached to its triggering flight, and the pre-check row prevents an event from repairing an already overdue arrival. The four methods evaluated in the matched A-only experiment are the original Khaled row, the endpoint split, the strict corrected legacy rows, and the ordered event-based formulation. Their builder identifiers are respectively `legacy_paper_c13`, `legacy_endpoint_split`, `legacy_corrected`, and `event_based`. A result is described as objective-matched only when both compared methods are optimal.

6.3.1. A-only timing versus planning horizon

We compared the four methods on the same corrected feasible instances with $|P| = 10$ and $h \in \{7, 15, 21, 30\}$ using unrestricted CPLEX 12.6 from `F:/Progs/IBM.ILOG.CPLEX.for.AMPL.v12.6-EAT/CPLEXamp.exe`. The instances use overnight returns and initial A/B hours chosen so that the seeded rotations are feasible under day/night maintenance semantics. The complete

measurements are in `results/tables/formulation_a_comparison_four_corrected_latest.csv`. Thus this table is a one-instance-per-horizon experiment, not a reproduction of Khaled et al.'s Table 6. The four input files contain 36 flights for $h = 7$ and 40 flights for each of $h \in \{15, 21, 30\}$, with $|P| = 10$, an A-check threshold of 270 minutes, initial A-hours of 90 minutes, 90-minute flight legs, and maintenance capacity 10 at MRO. They were generated by `src/generate_feasible_instances.py` with `-maintenance-families A`; the same JSON file is passed to all four builders for each horizon.

All four methods are optimal on every horizon in the corrected feasible family. All four methods have identical objectives here, confirming objective parity after the event-reset correction. The event-based method uses fewer variables than every legacy variant. It is faster than the paper and split variants on all four horizons and faster than corrected legacy on the three longer horizons; corrected legacy is slightly faster at $H = 7$. These results support a performance comparison under common feasibility semantics without implying universal runtime dominance.

6.3.2. A-only CPU scaling versus fleet size

The fleet-size experiment below supplies the matched four-method comparison that is needed to complement the horizon study.

6.3.3. Heavy A-only fleet-size scaling

To assess how model size changes with the number of aircraft, we use the A-only fleet-size suite in `results/tables/formulation_a_fleet_four_latest.csv`. These cases use $h = 7$, $|P| \in \{5, 10, 15, 20\}$, A checks only, overnight returns, and one common feasible instance at each fleet size. All four formulations are

Table 5: Four-method A-only comparison by planning horizon on the corrected feasible family. All reported runs are optimal. The method names identify the exact builder and C13 treatment.

h	Method	Status	Objective	CPU (s)	Wall (s)	Vars	Cons.
7	Legacy paper	optimal	17526	0.318	0.656	1720	3035
	Legacy split	optimal	17526	0.323	0.674	1720	3315
	Legacy corrected	optimal	17526	0.307	0.636	1810	3095
	Event-based	optimal	17526	0.311	0.647	980	4186
15	Legacy paper	optimal	19586	0.383	0.798	2520	4304
	Legacy split	optimal	19586	0.391	0.820	2520	5084
	Legacy corrected	optimal	19586	0.343	0.697	2660	4064
	Event-based	optimal	19586	0.325	0.668	1130	4380
21	Legacy paper	optimal	19570	0.380	0.790	2520	4304
	Legacy split	optimal	19570	0.397	0.847	2520	5084
	Legacy corrected	optimal	19570	0.346	0.704	2660	4064
	Event-based	optimal	19570	0.324	0.668	1130	4380
30	Legacy paper	optimal	19592	0.376	0.771	2520	4304
	Legacy split	optimal	19592	0.395	0.821	2520	5084
	Legacy corrected	optimal	19592	0.351	0.717	2660	4064
	Event-based	optimal	19592	0.327	0.673	1130	4380

run on the same JSON instance for each value of $|P|$: the Khaled paper row, legacy endpoint split, legacy corrected exact state, and ordered Event-based model. All 16 runs are optimal and objective-matched. The plots report active scalar variables and constraints counted directly from the built Pyomo models; the constraint plot is the requested dependency-size view.

Table 6 gives the exact CPU and wall-time values used in Figures 5 and 6. Values are in seconds; each group of four columns follows the method order in the first column.

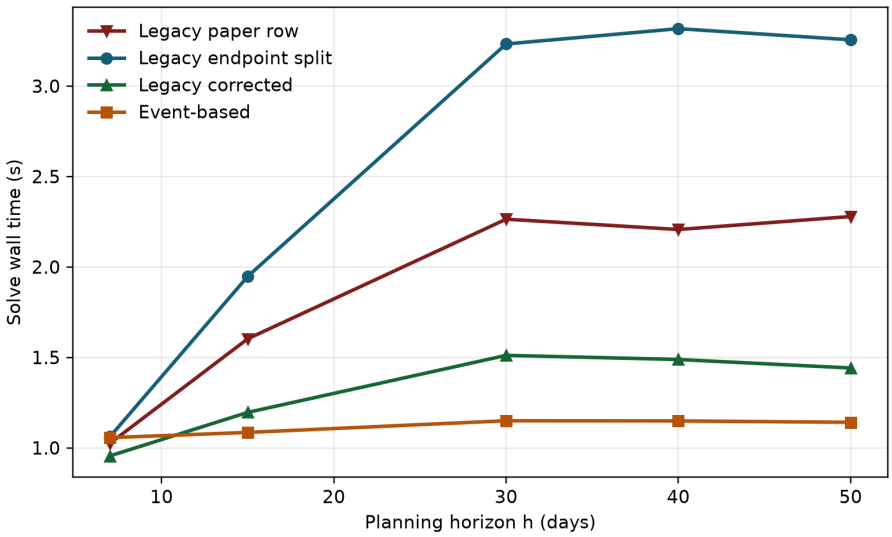

Figure 1: A-only solve wall time versus planning horizon for the four-method comparison at $|P| = 20$ in the extended suite.

6.4. Summary and limitations

The corrected type-A model rejects the cumulative-flight-time counterexample. The four-way horizon experiment shows that all formulations are feasible and optimal on the corrected generated family, with the explicit-state legacy and event variants improving on the legacy split in solve time. The event formulation also uses fewer variables and matches the legacy objectives on this family. The study does not claim a solver-independent runtime advantage.

6.4.1. Expanded four-method suite

To broaden the single-cell horizon and fleet-size snapshots above, we ran the dedicated four-method suite in `results/tables/four_method_suite.csv`. It contains 12 generated A-only performance cases on the Cartesian grid

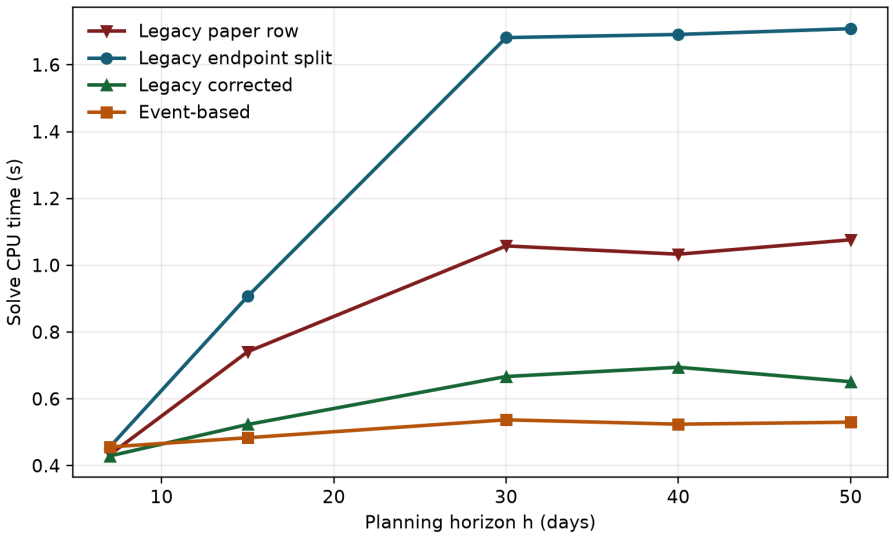

Figure 2: A-only solve CPU time versus planning horizon for the four-method comparison at $|P| = 20$ in the extended suite.

$|P| \in \{5, 10, 20, 40\}$ and $H \in \{7, 15, 30\}$, plus the two regression cases `c13_loophole_test` and `event_vs_legacy_clean_test`. Every performance case uses one common JSON instance for all four methods and all 48 performance runs are optimal. The aggregate statistics are in `results/tables/four_method_suite_summ`

The expanded suite confirms the focused comparisons: corrected legacy has the lowest mean CPU and wall time in this regenerated run, while the Event-based model uses approximately 63% fewer variables than the legacy models. Its constraint count is slightly higher than the legacy-paper count because its route and timestamp formulations replace compact day-indexed rows; this is a structural trade-off, not an omitted capacity condition. The C13 loophole regression has the expected verdict: the paper and split rows accept the fixed violating pattern, whereas the corrected state rejects it. The clean regression is optimal under all four methods.

Table 6: Heavy A-only fleet-size timing data used in Figures 6 and 7. Values are copied from `formulation_a_fleet_four_latest.csv`.

$\|P\|$	CPU time (s)				Wall time (s)			
	Paper	Split	Corrected	Event	Paper	Split	Corrected	Event
5	0.278806	0.284396	0.285962	0.279441	0.563580	0.570828	0.570250	0.585978
10	0.322849	0.323723	0.307218	0.314193	0.688894	0.672230	0.637101	0.654668
15	0.366002	0.374803	0.351611	0.372924	0.789226	0.827724	0.749238	0.805344
20	0.435350	0.458497	0.427633	0.474430	1.016204	1.062455	0.951910	1.069048

Table 7: Aggregate statistics over the 12-case A-only performance suite. Variables and constraints are active scalar Pyomo components; times are in seconds.

Method	Mean variables	Mean constraints	Mean CPU	Mean wall
Legacy paper	14 225.417	26 923.917	1.059189	2.414120
Legacy split	14 225.417	29 823.917	1.478338	3.040388
Legacy corrected	14 544.167	25 261.417	0.692639	1.609347
Event-based	5 279.167	27 767.500	0.818958	1.635593

6.4.2. Strengthened corrected-legacy implementation

We separately evaluated the equivalent implementation strengthening from Section 4.9; it is not substituted into the four-method table above, so those baseline results remain reproducible. The comparison uses the same 12 A-only performance cases, three repetitions per case and implementation, unrestricted CPLEX 12.6, and a 60-second limit. All 36 baseline/strengthened pairs have identical optimal objectives. The clean regression is also objective-matched, and both implementations reject the C13 loophole instance as infeasible.

The strengthened implementation reduces the mean generated constraint count by 37.61%, mean CPU time by 5.23%, and mean wall time by 5.90%.

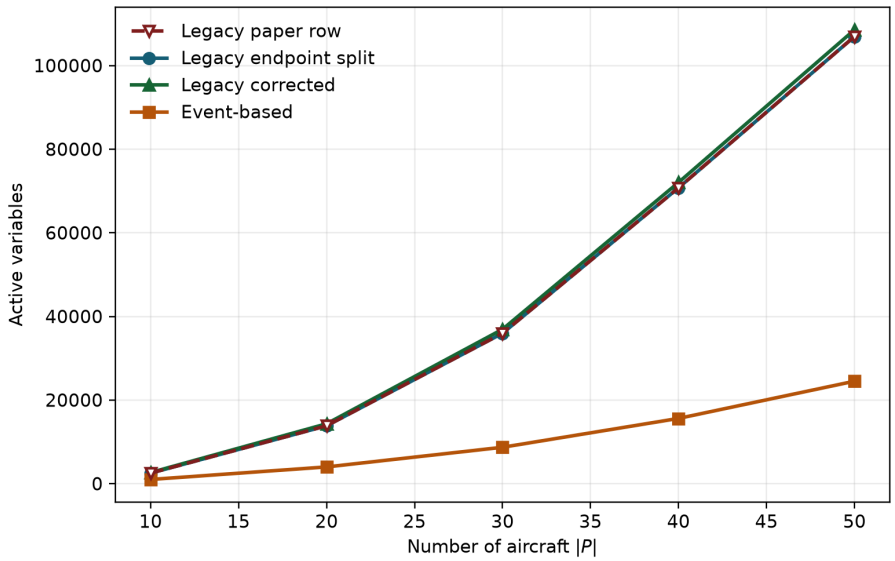

Figure 3: Active model variables versus fleet size at $H = 30$ in the extended four-method A-only suite.

It is faster in 22 of 36 paired CPU measurements and 25 of 36 paired wall measurements. The variable count is unchanged because both compared implementations contain the same type-A state variables. These measurements support the strengthening as an implementation improvement, not as a new mathematical formulation.

6.4.3. Optimized Event-based implementation

We next compared the baseline Event-based implementation, the equivalent reduction in Section 5.5, and the strengthened corrected legacy implementation on the same 12 performance cases and three repetitions. All 36 three-way groups are optimal and objective-matched. The optimized and baseline Event-based implementations also agree on both regression cases; the corrected legacy variant retains its intended infeasible verdict on the C13

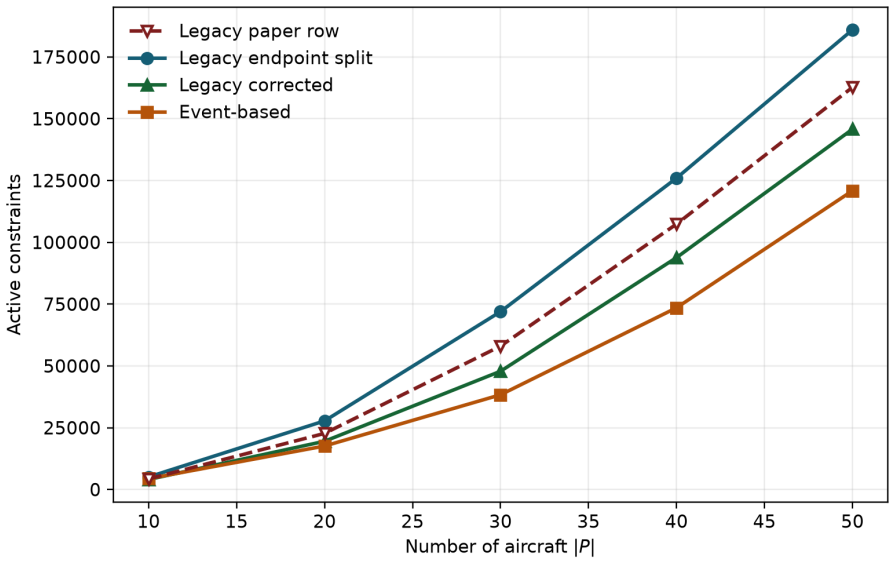

Figure 4: Active model constraints versus fleet size at $H = 30$ in the extended four-method A-only suite.

loophole case.

Relative to the baseline Event-based implementation, the optimized variant uses 5.68% fewer variables and 44.90% fewer constraints, with 26.79% lower mean CPU and 19.58% lower mean wall time. It is faster in 33 of 36 paired CPU measurements and 34 of 36 paired wall measurements. On this test grid it also improves on strengthened corrected legacy by 7.62% in mean CPU and 12.72% in mean wall time, while using 65.77% fewer variables and 2.94% fewer constraints. It wins 25 of 36 paired CPU and 25 of 36 paired wall comparisons against strengthened corrected legacy. These results demonstrate an implementation-level improvement for this instance family, not universal runtime dominance.

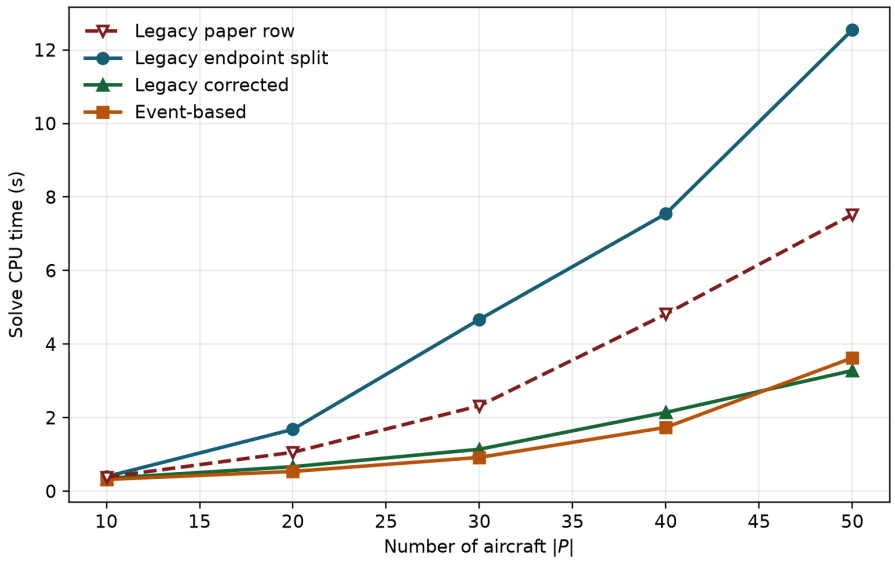

Figure 5: Solver CPU time versus fleet size at $H = 30$ in the extended four-method A-only suite.

6.4.4. Validation scope and required robustness checks

The A-only results are a controlled formulation comparison, not a final performance claim. The current evidence uses one deterministic generated case per (P, H) cell, a single maintenance-capable airport pattern, one set of maintenance parameters, and one unrestricted CPLEX 12.6 configuration. The four-method suite also contains two targeted regressions, but these are correctness checks rather than statistically independent performance samples. Consequently, the observed Event-based reductions in variables and average wall time should be interpreted as results for this instance family and solver configuration only.

A broader validation should repeat the same four-method comparison on multiple seeds and structurally different instances: hub-and-spoke and

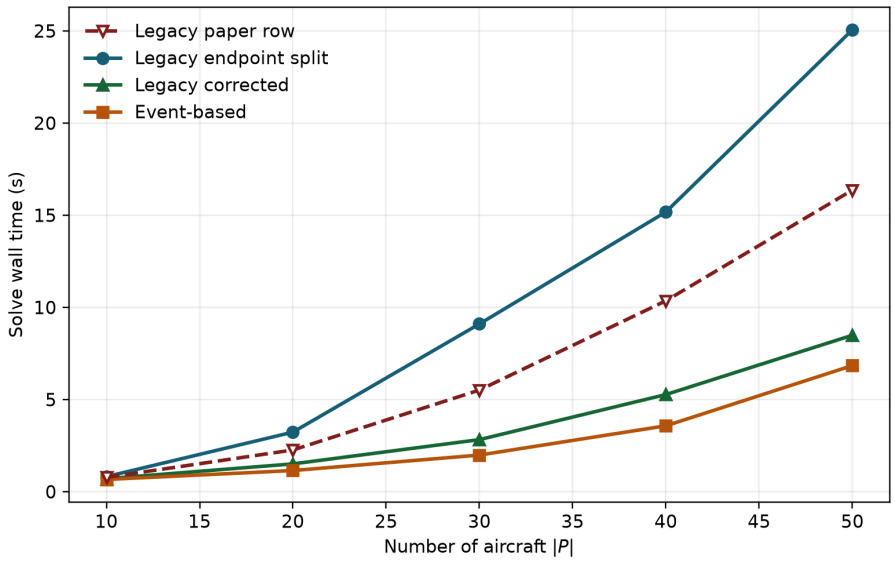

Figure 6: Solver wall time versus fleet size at $H = 30$ in the extended four-method A-only suite.

mixed-airport networks, sparse and dense connection graphs, different maintenance capacities, different A-check thresholds and durations, nonzero valid initial counters, and instances containing simultaneous departures and tight turn windows. The solver study should additionally repeat each case under CPLEX with and without presolve, vary the time limit and warm-start policy, and include at least one independent MILP solver where the model is supported. Each run should retain status, optimality gap, objective, active variable and constraint counts, build time, CPU time, wall time, and peak memory. Objective parity should be checked before timing comparisons are aggregated. These checks are necessary to separate formulation effects from instance, solver, and configuration effects.

As an initial configuration check, the representative four-case subset was

Table 8: Corrected-legacy implementation strengthening over 36 paired runs. Times are means in seconds.

Implementation	Variables	Constraints	Build	CPU	Wall
Corrected baseline	14 544.167	25 261.417	0.698785	0.659484	1.557829
Corrected strengthened	14 544.167	15 761.833	0.643380	0.624975	1.465910

Table 9: Event-based implementation optimization over 36 paired runs. Times are means in seconds.

Implementation	Variables	Constraints	Build	CPU	Wall
Corrected legacy strengthened	14 544.167	15 761.833	0.640321	0.619518	1.445313
Event-based baseline	5 279.167	27 767.500	0.582939	0.781733	1.568599
Event-based optimized	4 979.167	15 299.167	0.478636	0.572310	1.261526

also run with CPLEX cold and warm starts. Both configurations produced the same 15 optimal rows and one expected C13-loophole infeasibility; their performance summaries were close, with mean CPU times of $(0.521, 0.614, 0.414, 0.391)$ seconds for the paper, split, corrected, and Event-based methods in the cold run, and $(0.517, 0.614, 0.414, 0.391)$ seconds in the warm run. Attempts with the available CBC and HiGHS executables, and with the native CPLEX plugin, failed at the solver interface before producing model results. They are retained as diagnostic outputs, not treated as independent-solver evidence.

7. Conclusions and Future Work

This paper revisited the compact Tail Assignment Problem formulation of Khaled et al. (2018) and made three focused contributions. First, we showed

that the basic continuity and turn-time inequalities admit a simultaneous-departure corner case and closed it with explicit non-overlap constraints. Second, we proved that the paper's cumulative-flight-time row can relax a maintenance interval incorrectly and that the natural endpoint split is also insufficient. We provided an exact cumulative-state correction and verified the failure against the production constraint objects. Third, we developed a day-free event-based reformulation for type-A maintenance and an ordered prefix-state implementation.

The event formulation attaches maintenance to its triggering flight and propagates the flight-hour counter along the aircraft's selected route. In the four-method horizon experiment on the corrected feasible family, all four models were optimal. The three legacy variants had identical objectives, and the event-based model used the same maintenance-cost accounting. The explicit-state corrected legacy model was faster than the endpoint split on every tested horizon. These results demonstrate the need to separate feasibility correction from objective accounting; they do not support a solver-independent speedup claim.

Equivalent implementation strengthening reduced the corrected legacy constraint count substantially, and removing exact A-only redundancies from the Event-based implementation reduced both model size and runtime. In the paired 12-case experiment, the optimized Event-based implementation had lower mean CPU and wall time than strengthened corrected legacy, while preserving all tested objectives and regression verdicts. This ordering remains specific to the tested instances and CPLEX configuration.

These computational results are deliberately scoped. They use determin-

istic synthetic A-only instances, one instance per parameter cell, and a single unrestricted CPLEX configuration. The results establish objective parity on the tested family and expose model-size and timing differences among the four formulations; they do not establish a universal speedup across solvers, solver configurations, instance distributions, maintenance parameter sets, or larger benchmark families. Multiple seeds, structurally different networks, presolve and warm-start variants, and an independent MILP solver remain necessary for a broader performance conclusion.

Reproducibility.. The project contains the model implementations, deterministic instance generator, benchmark scripts, result tables, and plotting scripts needed to reproduce the reported snapshots. The legacy, event, and ordered A-only builders remain separately selectable so future comparisons can distinguish model semantics from implementation choices.

Future directions

- **Larger A-only benchmark families:** repeat the horizon and fleet-size experiments over multiple seeds and heterogeneous timetable structures, reporting solver variability and confidence intervals.
- **Solver and configuration robustness:** repeat the comparison with and without presolve, under different time limits and warm-start policies, and with an independently implemented MILP solver.
- **Integrated fleet assignment and routing:** extend the corrected compact model while retaining physically executable route constraints.

- **Robust and recoverable variants:** study uncertainty in flight times, disruptions, and maintenance availability under the exact state formulation.
- **Heuristic methods:** develop constructive and local-search methods for the corrected A-only model and compare them with the exact formulations on larger instances.

References

Barnhart, C., Belobaba, P., Odoni, A.R., 2003. Applications of operations research in the air transport industry. Transportation Science 37, 368–391.

Barnhart, C., Boland, N.L., Clarke, L.W., Johnson, E.L., Nemhauser, G.L., Shenoi, R.G., 1998. Flight string models for aircraft fleeting and routing. Transportation Science 32, 208–220. doi:10.1287/trsc.32.3.208.

Borndörfer, R., Dovica, I., Nowak, I., Schickinger, T., 2010. Robust tail assignment. Technical Report. Konrad-Zuse-Zentrum für Informationstechnik Berlin.

Clarke, L., Johnson, E., Nemhauser, G., Zhu, Z., 1997. The aircraft rotation problem. Annals of Operations Research 69, 33–46. doi:10.1023/A:1018945415148.

Clarke, L.W., Hane, C.A., Johnson, E.L., Nemhauser, G.L., 1996. Maintenance and crew considerations in fleet assignment. Transportation Science 30, 249–260.

Cordeau, J.F., Stojković, G., Soumis, F., Desrosiers, J., 2001. Benders decomposition for simultaneous aircraft routing and crew scheduling. Transportation Science 35, 375–388.

Desaulniers, G., Desrosiers, J., Dumas, Y., Solomon, M.M., Soumis, F., 1997. Daily aircraft routing and scheduling. Management Science 43, 841–855.

Froyland, G., Maher, S.J., Wu, C.L., 2013. The recoverable robust tail assignment problem. Transportation Science 48, 351–372.

Gopalan, R., Talluri, K.T., 1998. Mathematical models in airline schedule planning: A survey. Annals of Operations Research 76, 155–185.

Grönkvist, M., 2005. The tail assignment problem. Ph.D. thesis. Chalmers, Göteborg University.

Hane, C.A., Barnhart, C., Johnson, E.L., Marsten, R.E., Nemhauser, G.L., Sigismondi, G., 1995. The fleet assignment problem: Solving a large-scale integer program. Mathematical Programming 70, 211–232.

Haouari, M., Shao, S., Sherali, H.D., 2012. A lifted compact formulation for the daily aircraft maintenance routing problem. Transportation Science 47, 508–525.

Khaled, O., Minoux, M., Mousseau, V., Michel, S., Ceugniet, X., 2018. A compact optimization model for the tail assignment problem. European Journal of Operational Research 264, 548–557. doi:10.1016/j.ejor.2017.06.045.

Klabjan, D., 2005. Large-scale models in the airline industry, in: Desaulniers, G., Desrosiers, J., Solomon, M.M. (Eds.), Column Generation. Springer US, pp. 163–195.

Lacasse-Guay, E., Desaulniers, G., Soumis, F., 2010. Aircraft routing under different business processes. Journal of Air Transport Management 16, 258–263.

Lavoie, S., Minoux, M., Odier, E., 1988. A new approach for crew pairing problems by column generation with an application to air transportation. European Journal of Operational Research 35, 45–58.

Levin, A., 1971. Scheduling and fleet routing models for transportation systems. Transportation Science 5, 232–255.

Liang, Z., Chaovalitwongse, W.A., 2012. A network-based model for the integrated weekly aircraft maintenance routing and fleet assignment problem. Transportation Science 47, 493–507.

Liang, Z., Chaovalitwongse, W.A., Huang, H.C., Johnson, E.L., 2011. On a new rotation tour network model for aircraft maintenance routing problem. Transportation Science 45, 109–120.

Mercier, A., Cordeau, J.F., Soumis, F., 2005. A computational study of Benders decomposition for the integrated aircraft routing and crew scheduling problem. Computers & Operations Research 32, 1451–1476. doi:10.1016/j.cor.2003.11.013.

Minoux, M., 1984. Column generation techniques in combinatorial optimization, a new application to crew pairing problems, in: Proceedings of the AGIFORS Symposium, Strasbourg, France.

Sarac, A., Batta, R., Rump, C.M., 2006. A branch-and-price approach for operational aircraft maintenance routing. European Journal of Operational Research 175, 1850–1869.

Sherali, H.D., Bish, E.K., Zhu, X., 2006. Airline fleet assignment concepts, models, and algorithms. European Journal of Operational Research 172, 1–30.